Deephaven as an application framework: verified facts ledger

One source of truth for every claim in our docs, deck and code. Each entry says how we know it. **Verified** = observed by a test in this repo on Deephaven Community 42.6 (pip `deephaven-server`, native Windows, Java 17; earlier rungs also on the Docker image) or read in the installed 42.6 source. **Docs** = stated in Deephaven documentation, not tested here.

Scopes, applications and fields

#	Fact	Evidence
S1	`run_script` (Python client) and `runCode` (JS API) run in the worker's single global scope; a name defined by one call is visible to the next	Verified: `tests/probe_concepts.py`
S2	Application-mode scripts run in that same global scope (`globals() is __main__.__dict__` was True). An undeclared top-level table became a `scope` field. `global` is **not** needed in Community 42.6 Python (the docs still advise it)	Verified: `tests/probe_app.d/scoping.py`
S3	With the pip server, the launching script's `__main__` **is** the console scope: its own functions appear there, and a client's `run_script` could read **and modify** the host's module globals	Verified: `tests/probe_introspection.py`, `tests/embedded_thread_demo.py`
S4	Underscore-prefixed names are published as `scope` fields like any other; an underscore does not keep a table private	Verified: `tests/probe_concepts.py`
S5	A worker holds several applications: `scope` plus one per `*.app` file; a built-in app with no exports does not appear in the field list	Verified: `tests/probe_concepts.py`
S6	Two applications can each publish a field with the same name; each resolves to its own object	Verified: `tests/probe_concepts.py`
S7	Fields are any registered object type (tables, figures, deephaven.ui elements, partitioned/hierarchical tables, plugin objects)	Verified: field types listed by `ListFields`; plugin registration in server logs
S8	`ApplicationState` (`get_app_state()["name"] = obj`) publishes a field under the app's own id	Verified: `tests/probe_app.d/scoping.py`

Tickets and sharing

#	Fact	Evidence
T1	Ticket formats: scope `s/<name>`; application `a/<app id>/f/<field>`; export `e` + 4-byte number (5 bytes); shared `h` + random 16-byte id	Verified: `pydeephaven` 42.6 `ticket.py`; export ticket length measured
T2	An export lives until released or its session closes, and keeps its object alive meanwhile	Verified: `pydeephaven` source docstrings

#	Fact	Evidence
T3	`bind_table(name, table)` promotes a client export to a scope field that outlives the binding session	Verified: `tests/probe_concepts.py`
T4	A shared ticket dies when the **publishing** session closes; a copy another session fetched earlier also stops working	Verified: `tests/probe_concepts.py`
T5	URIs `dh+plain://host:port/scope/<name>` and `dh+plain://host:port/app/<id>/field/<name>` resolve via `deephaven.uri.resolve`; `local:///...` has no resolver. A resolved URI appears in the worker's graph as a Barrage source node, even when pointing at the same server	Verified: `tests/probe_concepts.py`, introspection report
T6	`pq://<PQ>/scope/<name>` and `remote_table` are Enterprise's cross-PQ forms; data is copied into the consumer	Docs

Runtime: clock, threads, contexts

#	Fact	Evidence
R1	The default update graph cycles every ~1 s (`target_cycle_ms` 1000); rows arriving between cycles are batched into the next one	Verified: rungs 1 and 4 (`examples/`), introspection report
R2	Rows pushed from Python waited up to ~1 s (650–1,000 ms) before a cycle picked them up	Verified: rung 4
R3	`merged_listen` delivers one callback per cycle across all watched tables, with `None` for tables that did not change; a table read inside the callback is consistent with that step	Verified: rung 4
R4	An exception in a listener stops it permanently while feeds keep flowing	Verified: rung 4
R5	Re-running setup code stacks listeners unless the old handle is stopped; background tasks that read lifecycle state from globals leak until the worker restarts	Verified: rungs 3 and 4
R6	Execution contexts are **per thread**. The thread that started the server holds the systemic context; on another thread, `get_exec_ctx()` returns a static-only context and creating a ticking table fails (`UpdateGraphConflictException`). Capture the context on the server's thread (or inside `run_script`) and pass it in	Verified: `tests/embedded_thread_demo.py`
R7	Server executors `serial` and `concurrent` exist; `submit_task("concurrent", fn)` ran Python work on a JVM thread	Verified: `tests/probe_runtime_features.py`
R8	A second, **event-driven** update graph can be built from Python. Tables built under a context bound to it (`withUpdateGraph`, `table_publisher(update_graph=...)`) do not move on their own: three pushes stayed invisible for 1.5 s, then one `requestRefresh()` delivered them as one checkpoint (`A: 600` = 100+200+300)	Verified: `tests/probe_event_driven_clock.py`

#	Fact	Evidence
R9	The embedded server can be started on a **non-main thread**; the host's asyncio loop kept running during startup but at about half speed (89 ticks of a 50 ms sleep in 9.4 s)	Verified: `tests/embedded_thread_demo.py`
R10	In-process (embedded) use works without a client: tables built from the host, a listener bridged into asyncio via `call_soon_threadsafe`	Verified: `tests/embedded_thread_demo.py`
R11	GIL contention is real: while the host burned CPU in pure Python, median listener lag rose from 36.6 ms to 74.8 ms (max 41 → 87 ms)	Verified: `tests/embedded_thread_demo.py`
R12	Only one server per process: a second `Server(...).start()` fails with "Cannot create more than one instance of the server"	Verified: `tests/embedded_thread_demo.py`
R13	The host process exited promptly once its main coroutine returned, with the server on a daemon thread	Verified: `tests/embedded_thread_demo.py`
R14	Hosting the server as a subprocess works: up in 10.9 s, logic deployed with `run_script`, read at our own checkpoints; on Windows `terminate()` is a hard kill (exit code 1)	Verified: `tests/host_service_demo.py`
R15	Multiple update graphs per worker, each bound to an execution context	Docs (event-driven case Verified, R8)

Graph, cost and introspection

#	Fact	Evidence
G1	Every table operation and listener is a performance entry; Python callbacks were the costliest nodes (~195 ms per interval each) against ~80 ms for a rolling `update_by` and 3–36 ms for other Java operations	Verified: `examples/05_introspection.py`, `examples/pretty_graph.py`
G2	The ancestors log records dependencies only for nodes created after it was first opened; sources (no ancestors) appear only in the update performance log	Verified
G3	`ancestor_svg` / `ancestor_image` need the Graphviz `dot` binary, absent from the stock Docker image	Verified
G4	`merged_listen` creates one internal "Python Wrapped Listener recorder" per watched table; joins create `leftTable:` / `rightTable:` helper nodes; `update_by` creates per-bucket nodes	Verified: introspection report
G5	Java threads that call into Python appear in `threading.enumerate()` as `java-...` entries	Verified
G6	`introspect/dh_introspect.py` + client run end to end: field registry, scope objects, graph with cost, inputs, listeners vs handles, unreachable nodes (client exports), Python threads	Verified: `tests/probe_introspection.py`

Platform and tooling

#	Fact	Evidence
P1	The pip server runs natively on Windows (docs say WSL 2 is required) given Java 17+ in `JAVA_HOME`, the JVM DLL folders on `PATH`, and relative web-storage directories (otherwise: `jimage.dll` crash; "Invalid path: /layouts")	Verified: `server/run_server.py`
P2	`pydeephaven` 42.6 breaks with `bitstring` 5; pin `bitstring<5`	Verified
P3	React route: pin `@deephaven/jsapi-types` ~41.11 with the 1.3x UI packages; shim `@paciolan/remote-module-loader` under Vite; `IrisGrid` takes no `dh` prop	Verified: `react-client/`
P4	deephaven.ui `use_row_data` returns pandas-typed values (longs as floats)	Verified
P5	Formulas recompute only when their input columns change	Verified: rung 3
P6	Ten Deephaven gRPC services (Session, Console, Table, Application, Object, InputTable, PartitionedTable, HierarchicalTable, Storage, Config) plus Arrow Flight carrying Barrage	Verified: `deephaven_core.proto` stubs

Extension points and replay

#	Fact	Evidence
X1	A user-compiled Java class on `Server(extra_classpath=[jar])` is callable from formulas by fully qualified name (`com.acme.Risk.dv01(Qty, Px, 7.5)`) or after `getQueryLibrary().importStatic(<Class>)`	Verified: `tests/probe_framework_constructs.py`
X2	`query_library.import_static("com.acme.Risk")` raised ClassNotFound for that jar although `jpy.get_type` loaded it	Verified: same
X3	Python functions work in formulas; float hints give a `double` column; no hints give a `PyObject` column; `int` / `np.int64` hints are rejected on native Windows (numpy reports int64 as `q`, Deephaven accepts `l`)	Verified: same, and `deephaven/_udf.py`
X4	`function_generated_table(fn, refresh_interval_ms=200)` ran 3 times in 1.5 s (paced by the ~1 s cycle)	Verified: same
X5	`ring_table(t, 5)` held exactly the last 5 rows; a keyed `input_table` upserts by key and deletes by key	Verified: same
X6	`TableReplayer` released rows by timestamp at wall-clock pace (20 rows over ~4 s) through unchanged graph code; `now()` stayed wall-clock time	Verified: same
X7	On an event-driven graph, `requestRefresh()` returned only after the step finished (200/200); 200 slices took 0.137 s	Verified: same
X8	One `build_engine(quotes, limits, desks)` ran live (5.76 s, 4 router calls) and as an event-driven backtest (0.46 s, 28 calls) on the same 900 quotes, with an identical final book and desk risk (33 orders each)	Verified: `examples/06_trading_engine.py`

#	Fact	Evidence
X9	Enterprise replay queries (`ReplayScript`) swap in a simulated clock (`today`, `timeTable`) and redirect `db.live_table` to history, with speed and sorted-replay settings	Docs
X10	Custom Java sources implement `StreamPublisher` and feed `StreamToBlinkTableAdapter`	Docs (Javadoc)
X11	On native Windows, `parquet.read` / `parquet.write` reject `C:\...` paths (URISyntaxException); `file:///` URIs work, including key=value partitioned directories	Verified: `tests/test_dhfw_community.py`
X13	The rung-6 engine rebuilt on `dhapp` (3 stages, 1 action) gave an identical book and risk live (5.7 s) and in backtest (150 one-second buckets, 1.3 s), matching rung 6's hand-written numbers exactly	Verified: `examples/07_app_framework.py`
X14	A supervised action that raises is recorded in the health table (1 error in 1 call) and does not stop other actions; test mode steps deterministically (a repeated target made no new order)	Verified: same
X15	`dhapp.ui.dashboard(app)` rendered live in the browser; editing a keyed input table's cell in the grid changed downstream risk at the next cycle (Tech utilization 135.1% to 82.6%, breach cleared)	Verified: `examples/mm_app.d/mm_live.py`, browser
X12	The `dhfw` wrapper's Community emulations pass 12/12 checks (historical and user tables, live append, system-table logging, registered feeds, replay mode, input tables, remote_table and `pq://`, controller imports, ACL no-op, explicit unsupported calls). Its Enterprise forwarding is untested	Verified (Community only): same

Corrections to earlier claims

Earlier claim	Correction
App-mode scripts need `global` to reach the console scope	Not in Community 42.6 Python (S2)
An underscore keeps a name private	Underscore names are published (S4)
Export tickets look like `e/<n>`	`e` + 4 binary bytes (T1)
A shared ticket can outlive its publisher	It dies with the publishing session (T4)
One clock per worker	One by default; more can be created, including event-driven (R8, R15)
No built-in Solace or kdb connector	Enterprise has Solace streaming import (Docs); kdb still none known
Unknown whether PQ scripts can come from Git	Enterprise supports Git-sourced PQ scripts (Docs)
Native Windows unsupported	Works with three workarounds (P1)
`get_exec_ctx()` works on any thread	Only on a thread that already holds a real context (R6)

Unverified: open questions from the audit

An audit compared every artifact against this ledger. It flagged these claims, but the checks on them never ran. Treat each one as **open**: it may be overstated, imprecise or wrong. The **Status** column records any fix made since.

#	Claim (where)	Why it is in doubt	Status
U1	T4: a copy another session fetched earlier also stops working (ledger)	`fetch_table` may only re-reference the shared ticket, not make a copy; then the test shows one ticket dying, not a copy	Open
U2	R11: median listener lag rose from 36.6 to 74.8 ms under host CPU load (ledger, Anatomy, deck)	One run with only 4 idle and 6 busy callbacks; the busy window includes a 1.5 s drain. The direction is plausible, the size is not established	Open: re-run with more samples
U3	R9: the host loop ran at about half speed during startup (89 ticks in 9.4 s) (ledger, Anatomy)	No idle baseline; on Windows a 50 ms asyncio sleep often takes longer anyway	Open: measure a baseline
U4	R2 / Findings: latency is "bounded below by the cycle"; pushed rows wait ~1 s	`MaxBufferedMs` is the oldest row's wait. The cycle caps the wait from above; a row pushed just before a cycle waits almost nothing	Open: reword as "up to one cycle"
U5	S7: fields are any registered object type (ledger)	Only tables and a deephaven.ui element were seen in `ListFields`; figures, partitioned and plugin objects were not tested	Open
U6	G6 / Findings: "2 Python listeners matching 2 handles, using 34% of engine time"	The saved report says 46% (174 of 381 ms) and 4 active listeners	Open: quote the report
U7	Findings summary: each cycle is consistent across **all tables in the worker**; cadence is set per worker	True only for the default graph; tables on an event-driven graph keep their own steps (R8)	Open: same fix as in Concepts and Anatomy
U8	Findings: "a worker can add update graphs" marked Verified	Only the event-driven case is verified (R8); the general claim is Docs (R15)	Open
U9	Findings: "Three realistic shapes"	Four numbered shapes follow	Open
U10	Findings: process lifecycle ("CPython embedded through jpy...") marked Verified	No ledger entry; describes the Docker worker, not the pip server, where Python starts first	Open
U11	Findings: scheduled stops are hard stops; failover can run two copies	Enterprise behaviour stated with no Docs marker	Open: mark Docs or confirm
U12	Anatomy: only the script session and the server's thread hold the real context	The test checked one other thread; listeners on update threads also run under a context	Partly fixed (Anatomy reworded)
U13	T2: export lifetime "Verified" from pydeephaven docstrings	A docstring is documentation, not an observation	Open: test it or mark Docs

#	Claim (where)	Why it is in doubt	Status
U14	G2, G3, G5, P2, P4 marked Verified with no test named	Evidence exists for some (G5 in the report), not recorded here	Open: name the evidence
U15	`e/7` and single-update-graph wording in Concepts; "two workarounds" in the README	Contradicted T1, R8 and P1	Fixed

Also open in the introspection tool (code, not claims):

- **Fixed:**
- `update_by` bucket nodes were not folded as internal
- client: cleanup now runs even when a step fails
- client: a stale report is no longer read
- client: the field filter keeps other apps' `dhi_*` fields
- client: printing on Windows no longer fails before the JSON is written
- client: help and docstring wording
- **Open:**
- **Nodes miscounted:** stopped listeners still count as reachable roots, and listeners without a description look "stacked".
- **Parentless ops listed as inputs:** an op or sink with no parents shows up under INPUTS.
- **Missing descriptions:** `merged_listen` handles report no description.
- **Some time tables misread:** a `time_table` with a start time is classified as an operation, not a source.
- **Port list too broad:** listening ports outside a container include the whole host.
- **Library list wrong in places:** on Python 3.9 every standard library shows as non-platform, and the pip server shows as non-platform too.
- **DataFrames:** pandas DataFrame fields are typed incorrectly, and underscore-named DataFrames are skipped.
- **No opt-out:** opening the performance logs can't be skipped.